

컴퓨터 구조 이론 및 실습 보고서

<아두이노 기초 프로그래밍>



동의대학교
DONG-EUI UNIVERSITY

학과 : 컴퓨터소프트웨어 공학과

과목 : 컴퓨터 구조와 이론

담당 교수님 : 김성우 교수님

학번 : 20212979

이름 : 임진호

목 차

I . 준비 사항	3-8
-----------------	-----

II . 실습	9-21
---------------	------

III . 결과	22-23
----------------	-------

1. 준비사항

▶ 아두이노 프로그램 구조

- 아두이노 프로그램은 C/C++을 기반으로 만들어졌다.
- main() 함수가 없고 setup() 과 loop() 함수로 구성되어 있다.

A. void setup()

- 초기에 한번만 실행
- 변수 초기화, 핀 모드 설정 등 작업
- 입력 인자와 반환값 없음

B. void loop()

- 반복적으로 실행
- 다양한 실제 작업 수행
- 입력 인자와 반환값 없음

C. Arduino의 장점

- 싼 가격과USB 지원
- 쉽고 간단한 프로그래밍 환경
- 오픈소스 하드웨어
- 오픈소스 소프트웨어

▶ 다양한 아두이노 함수

A. 디지털 입출력

- pinMode(pin, mode)

지정한 핀을 입력 또는 출력모드로 설정

pin - 설정하려고하는핀번호

mode - 설정모드: INPUT, OUTPUT or INPUT_PULLUP

반환값없음-void

- digitalWrite(pin,value)

출력설정핀에HIGH 또는LOW 값을출력

pin - 출력할핀번호

value - HIGH (5V 또는3.3V 전압) 또는LOW (0V 전압)

반환값없음: void

- digitalRead(pin)

입력설정핀으로부터디지털값을읽음

pin - 입력받을핀번호

HIGH 또는LOW 값을반환

B. 아날로그 입출력

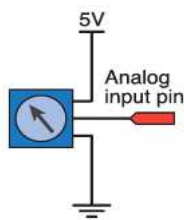
-analogRead(pin)

아날로그입력핀의전압을읽음

pin-아날로그입력채널을정의하는정수: 0 ~ 5

(pin은실제로 A0, A1, A2, A3, A4, A5 를가리킴)

0 부터1023 까지의정수를반환



```
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  int reading;  
  reading = analogRead(A0);  
  Serial.println(reading);  
}
```

-가변저항 값 읽는 프로그램-

-analogWrite(pin, value)

PWM 신호를디지털핀으로출력

(PWM 신호는약490Hz)

value-듀티사이클의값. 0 (0%) 에서255(100%) 사이의값

C. 고급 입출력

- tone(pin, frequency, duration=0)

50% 듀티사이클과지정주파수를가진구형파를 출력

단음재생용

부저나스피커연결사용가능

duration-지속시간을나타내는값. 밀리초단위

-noTone(pin)

tone 함수에의한구형파출력을멈춤

-shiftOut(dataPin, clockPin, bitOrder, value)

지정된값을비트단위로데이터핀으로출력

clockPin-한비트출력후데이터출력을알리는펄스출력핀

bitOrder-비트출력순서. MSBFIRST 또는LSBFIRST

-ShiftIn(dataPin, clockPin, bitOrder)

비트단위로데이터를입력받음

clockPin-HIGH 상태후입력이되며, 입력완료후LOW 로바뀜

-pulseIn(pin, value, timeout = 100000L)

지정된핀으로부터HIGH 또는LOW 펄스를읽음

timeout-펄스의시작을기다리는시간. 마이크로초단위

D. 시간함수

-unsigned long millis(void)

실행중인프로그램이시작된이후경과시간을밀리초단위로반환

-unsigned long micros(void)

실행중인프로그램이시작된이후경과시간을마이크로초단위로반환

-delay(unsigned long ms)

지정한시간만큼프로그램실행을중단

ms-밀리초단위의지연시간

-delayMicroseconds(unsigned long us)

지정한시간만큼프로그램실행을중단

us-마이크로초단위의지연시간

E. 시리얼 통신 함수

-> Serial 클래스 제공

-void begin(speed)

인자speed 로직렬포트초기화. Typical speed is 9600

-size_t print(value)

직렬포트로value값을보냄

value는숫자또는문자열, newline 추가하지않음

-println(value)

직렬포트로value값을보낸후에newline (\n) 추가

value는숫자또는문자열

-size_t write(uint8_t ch)

직렬포트로바이트데이터출력

-int available(void)

직렬포트로수신한데이터의바이트수반환

-int read(void)

직렬포트로수신한첫번째바이트데이터또는-1반환

수학함수 : abs(), constrain(), map(), max(), in(), pow(), sq(), sqrt()

문자함수 : isAlpha(), isAlphaNumeric(), isAscii(), isControl(), isDigit(), isGraph(), isHexadecimalDigit(), isLowerCase(), isPrintable(), isPunct(), isSpace(), isUpperCase(), isWhitespace()

난수함수: random(), randomSeed()

▶ 직렬 통신

직렬통신은 시리얼 통신이라고도 부른다. 직렬통신은 데이터를 1비트씩 순차적으로 전송하는 방식의 통신방법이다. 데이터가 한 번에 하나씩 전송되기 때문에 간단하고 하드웨어 비용이 적게들며, 먼거리 통신에도 적합하다.

특징

데이터 전송방식

-한 번에 1비트씩 데이터를 전송

-송신측과 수신측에 동일한 속도로 데이터를 주고받아야 한다.

속도

-보드레이트(Baud Rate)로 전송 속도를 결정합니다. 일반적으로 사용되는 값은 9600, 115200 등입니다

직렬통신의 종류

1. 동기식 통신

- 데이터와 클럭 신호를 동시에 전송

2. 비동기식 통신

데이터와 클럭 신호를 별도로 전송하지 않고, 송신측과 수신측이 보드레이트를 미리 설정하여 동작한다.

장점 : 간단한 연결, 하드웨어 자원이 적게 소모됨, 먼거리 데이터 전송 가능,

단점 : 속도가 상대적으로 느림, 동기화 필요

▶ 아날로그 입출력과 PWM

아날로그 입력은 ADC(Analog Digital Converter)을 이용한다

ADC는 실생활에서 연속적인 아날로그 신호를 측정해 그 신호를 컴퓨터로 입력해 디지털로 변환 한다.

아날로그 출력은 DAC대신 PWM을 활용한다.

PWM은 디지털 신호의 펄스를 이용해 구형파의 듀티비를 조절함으로써 평균 전압을 제어한다.

아두이노에서 아날로그 입출력이 가능한 핀은 정해져있다.

아날로그 입력은 A0~A5를 이용해 입력하고

Arduino 보드에는 ~3,~5,~6,~9,~10,~11핀이 출력가능한 핀이다.

▶ 아두이노 어셈블리 프로그래밍

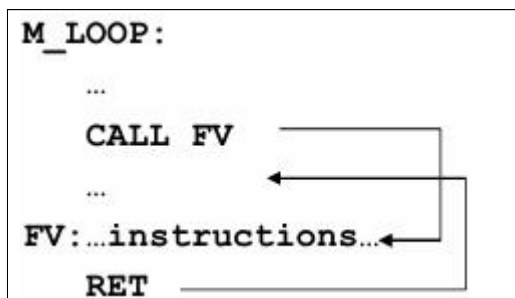
- 어셈블리란 C언어, 자바와 같은 고급언어와 다르게 기계어와 대응되는 저급언어이다. CPU 종류에 따라 고유한 어셈블리어를 사용한다.

무한루프

```
M_LOOP:
...instructions...
jmp M_LOOP
```

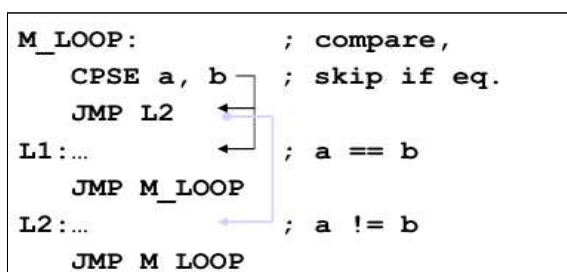
서브루틴

```
M_LOOP:
...
CALL FV
...
FV:...instructions...
RET
```

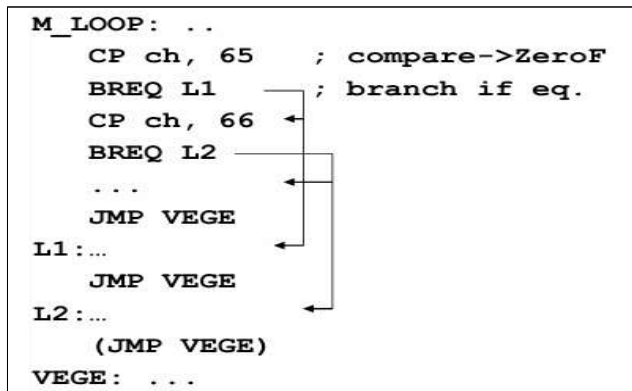


조건 분기

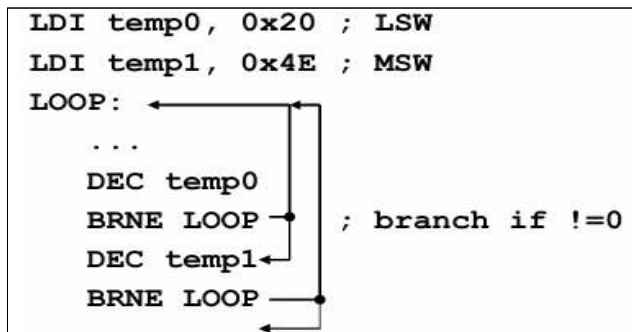
```
M_LOOP:                ; compare,
    CPSE a, b           ; skip if eq.
    JMP L2              ;
L1:...                  ; a == b
    JMP M_LOOP          ;
L2:...                  ; a != b
    JMP M_LOOP
```



switch/case

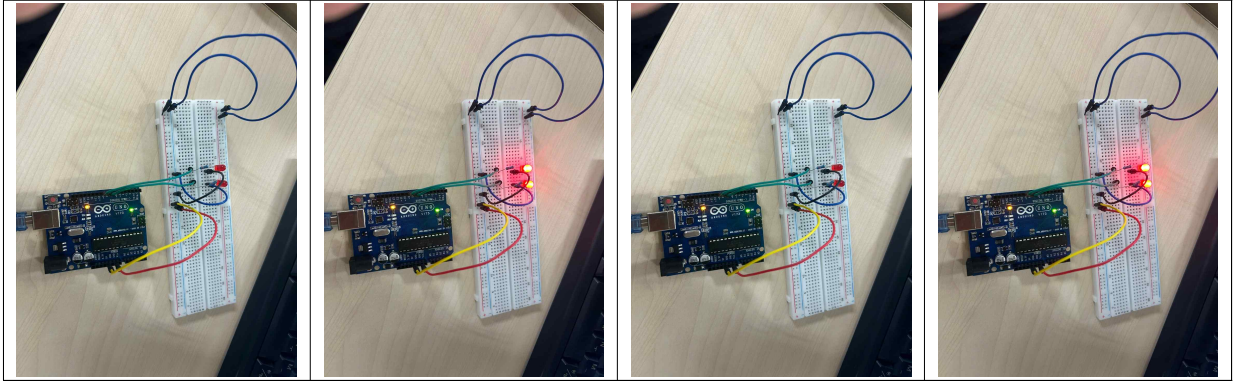


긴 for루프(1바이트 이상)



2. 실습

▶ LED 깜빡이기 회로 구성 및 결과

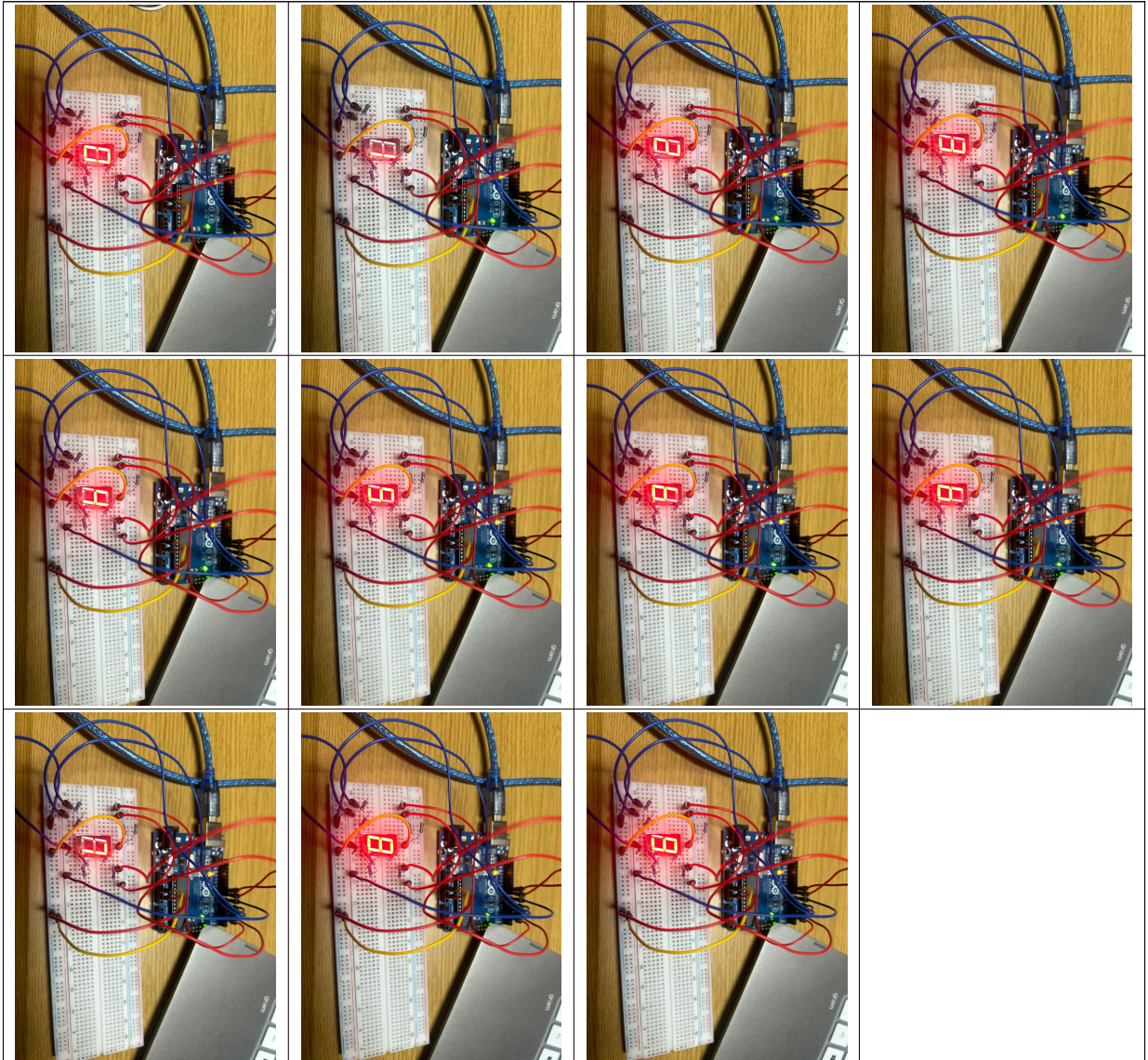


코드

```
int led1 = 8;
int led2 = 9;
void setup()
{
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
}
void loop()
{
  digitalWrite(led1, HIGH);
  digitalWrite(led2, HIGH);
  delay(500);
  digitalWrite(led1, LOW);
  digitalWrite(led2, LOW);
  delay(500);
}
```

delay()에서 괄호안의 500이란 수치를 더 큰 숫자를 넣게되면 깜빡거리는 템포가 늦어지고 숫자가 작아질수록 더 많이 깜빡거린다

▶ 7세그먼트



코드

```
int ON = HIGH;
int OFF = LOW;
int numerals[10][8] = {
  {OFF, ON, ON, ON, ON, ON, ON, ON}, // 0
  {OFF, OFF, OFF, OFF, OFF, ON, ON, OFF}, // 1
  {ON, OFF, ON, ON, OFF, OFF, ON, ON}, // 2
  {ON, OFF, OFF, ON, OFF, ON, ON, ON}, // 3
  {ON, ON, OFF, OFF, OFF, ON, ON, OFF}, // 4
```

```

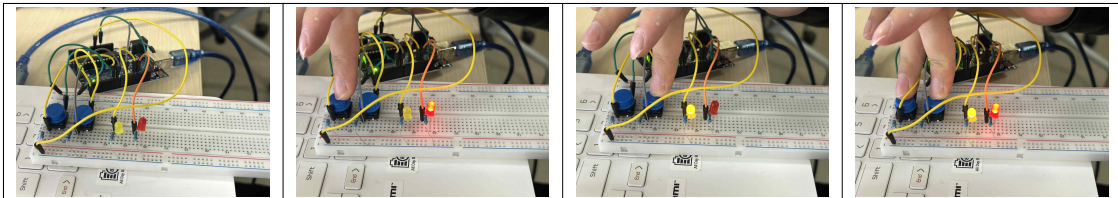
{ON, ON, OFF, ON, OFF, ON, OFF, ON}, // 5
{ON, ON, ON, ON, OFF, ON, OFF, OFF}, // 6
{OFF, OFF, OFF, OFF, OFF, ON, ON, ON}, // 7
{ON, ON, ON, ON, ON, ON, ON, ON}, // 8
{ON, ON, OFF, ON, ON, ON, ON, ON}, // 9};
int pins[] = {2, 3, 4, 5, 6, 7, 8, 9};
void setup() {
  for (int i=0; i < 8; i++) {
    pinMode(pins[i], OUTPUT);
  }
  void loop() {
    for (int i = 0; i < 10; i++) {
      for (int j = 0; j < 8; j++) {
        digitalWrite(pins[j], numerals[i][j]);
      } delay(1000); }}

```

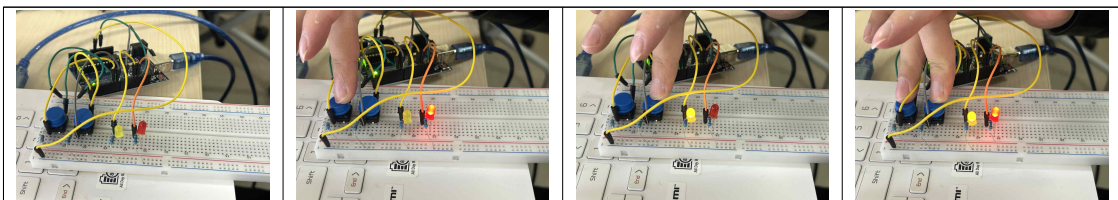
>>정수형 2차원 배열을 사용하여 첫 번째 인덱스때 0을표현~마지막 인덱스에 9를 표현하여 0~9의 숫자가 표현되게 만드는 코드입니다.

▶ 실습 3번 회로 구성 및 결과

1. 프로그래밍 x led제어



2. 프로그래밍 o led제어



코드

```

int led1 = 7;
int led2 = 6;
int key1 = 13;
int key2 = 12;
void setup()

```

```

pinMode(led1, OUTPUT);
pinMode(led2, OUTPUT);
pinMode(key1, INPUT);
pinMode(key2, INPUT);

void loop()

if (digitalRead(key1) == HIGH)
digitalWrite(led1, HIGH);
else
digitalWrite(led1, LOW);
if (digitalRead(key2) == HIGH)
digitalWrite(led2, HIGH);
else
digitalWrite(led2, LOW);
delay(100);
}

```

▶ 실습 5번 USB 포트를 이용한 시리얼 통신



```

void setup()

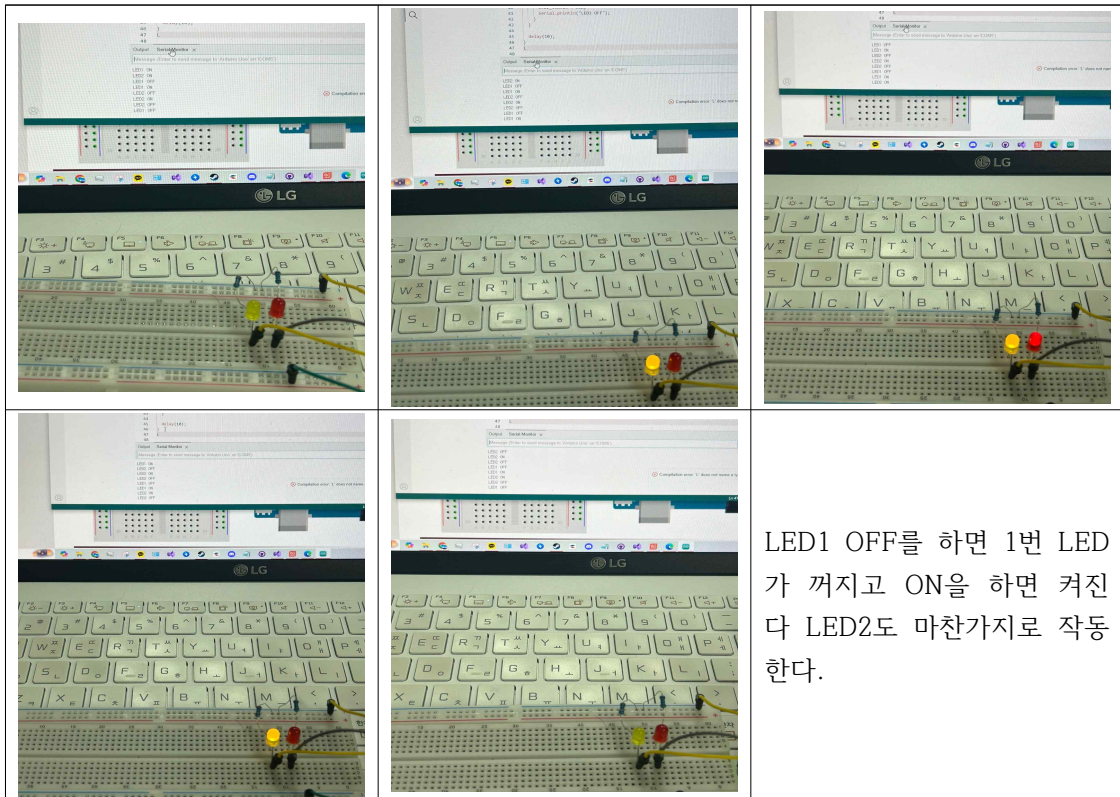
Serial.begin(9600);

void loop()

char read_data;
if (Serial.available())
read_data = Serial.read();
Serial.print(read_data);
delay(10);

```


► 5-2



LED1 OFF를 하면 1번 LED
가 꺼지고 ON을 하면 켜진
다 LED2도 마찬가지로 작동
한다.

```

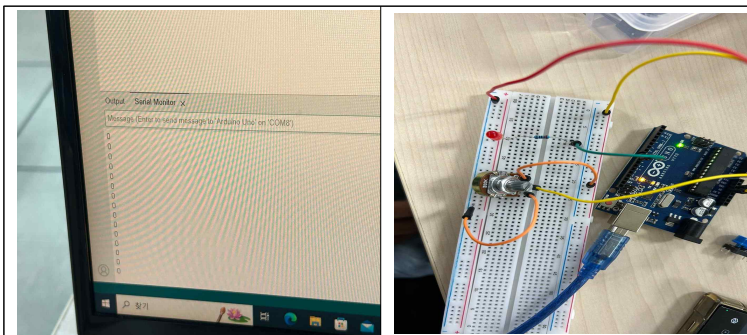
int led1 = 7;
int led2 = 6;
int led1_status = LOW; // LED1 상태
int led2_status = LOW; // LED2 상태
void setup()
{
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  digitalWrite(led1, LOW);
  digitalWrite(led2, LOW);
  Serial.begin(9600);
}
void loop()
{
  char read_data;
  if (Serial.available())
  {
    read_data = Serial.read();
    if( read_data == '1' && led1_status ==
  
```

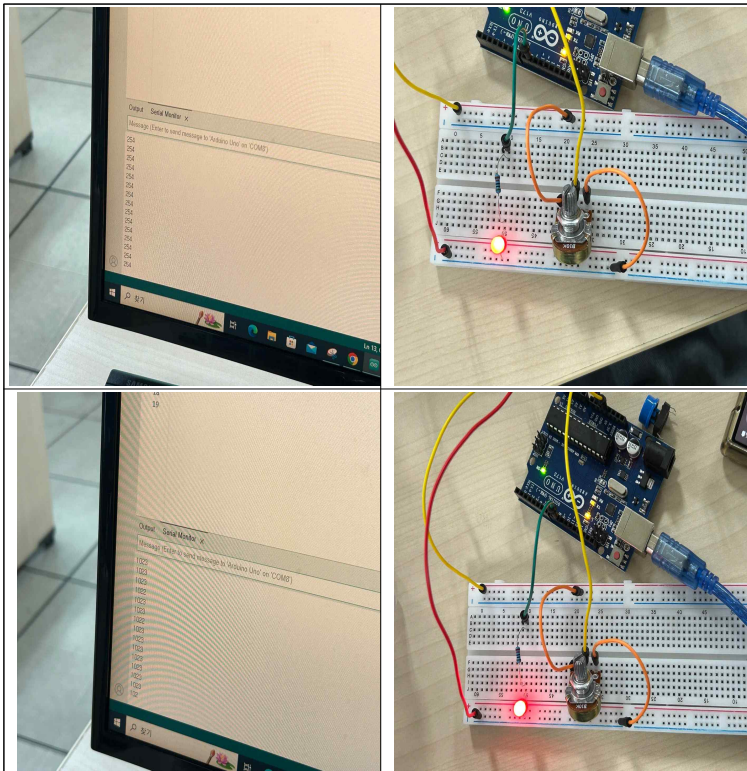
```

LOW)
{
  digitalWrite(led1, HIGH);
  led1_status = HIGH;
  Serial.println("LED1 ON");
}
else if( read_data == '1' && led1_status
== HIGH )
{
  digitalWrite(led1, LOW);
  led1_status = LOW;
  Serial.println("LED1 OFF");
}
}
if( read_data == '2' && led2_status ==
LOW)
{
  digitalWrite(led2, HIGH);
  led2_status = HIGH;
  Serial.println("LED2 ON");
}
else if( read_data == '2' && led2_status
== HIGH )
{
  digitalWrite(led2, LOW);
  led2_status = LOW;
  Serial.println("LED1 OFF");
}
}
delay(10);

```

▶ 가변저항을 이용해 LED밝기를 조절하는 회로





```
int sensorPin = A0; // select the input pin for the potentiometer
int led = 9; // the pin that the LED is attached to
void setup()

pinMode(led, OUTPUT);
Serial.begin(9600);

void loop()

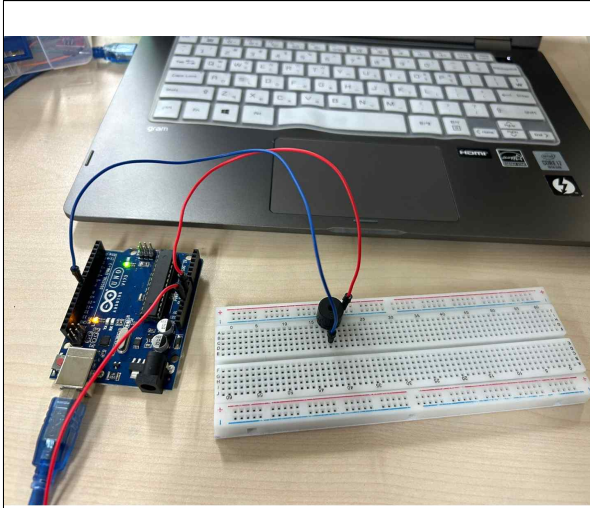
int sensorValue = 0;
sensorValue = analogRead(sensorPin);
analogWrite(led, sensorValue/4);
Serial.println(sensorValue);
delay(10);
```

▶ 코드 설명 :

- 0일땐 LED가 꺼지고 1023일 때 LED가 최대밝기로 켜집니다.

가변저항과 PWM의 범위를 맞춰 주기위해 가변저항을 4로 나누어 범위를 같게 만들어준다
사진에는 0과 254, 1023으로 불이 꺼지고 약하게 켜지고 강하게 켜지게 표현하였다
(가변저항의 범위는 0~1023이고 PWM의 범위는 0~255 이다)

▶ 8. 펄스(PWM)로 부저 연주하기



작은별 변주곡

by MUEIC

```
int buzzer = 8;
int tones[] = {262, 294, 330, 349, 392, 440, 494};
// C(도),D(레),E(미),F(파),G(솔),A(라),B(시)
void setup(){
  pinMode(buzzer, OUTPUT);
}
void loop(){
  for (int i=0; i<2; i++){ //도도
    tone(buzzer, tones[0],500);
    delay(500);
  }
  for (int i=0; i<2; i++){ //솔솔
    tone(buzzer, tones[4],500);
    delay(500);
  }
  for (int i=0; i<2; i++){ //라라
    tone(buzzer, tones[5],500);
    delay(500); }
  tone(buzzer, tones[4],800); //솔
  delay(800);
  for (int i=0; i<2; i++){ //파파
    tone(buzzer, tones[3],500);
    delay(500);
  }
  for (int i=0; i<2; i++){ //미미
    tone(buzzer, tones[2],500);
```

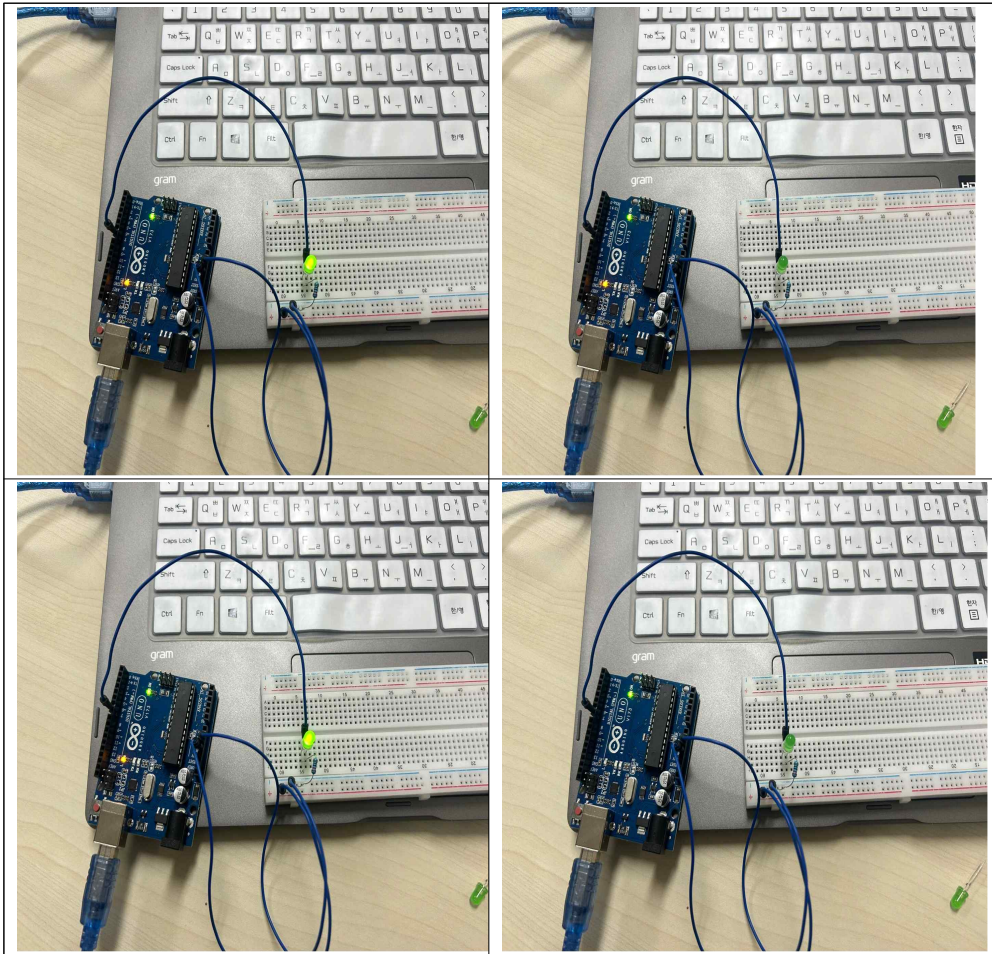


```

    delay(500);
}
    for (int i=0; i<2; i++){ //레레
        tone(buzzer, tones[1],500);
        delay(500);
    }
tone(buzzer, tones[0],800);
delay(800);
for (int i=0; i<2; i++){ //솔솔
    tone(buzzer, tones[4],500);
    delay(500);
}
for (int i=0; i<2; i++){ //파파
    tone(buzzer, tones[3],500);
    delay(500);
}
    for (int i=0; i<2; i++){ //미미
        tone(buzzer, tones[2],500);
        delay(500);
    }
tone(buzzer, tones[1],500);
    delay(1000);
    for (int i=0; i<2; i++){ //솔솔
        tone(buzzer, tones[4],500);
        delay(500);
    }
for (int i=0; i<2; i++){ //파파
    tone(buzzer, tones[3],500);
    delay(500);
}
    for (int i=0; i<2; i++){ //미미
        tone(buzzer, tones[2],500);
        delay(500);
    }
tone(buzzer, tones[1],500);
    delay(500);
noTone(buzzer);
delay(1000);
}

```

▶ 9. BLink 어셈블리 버전 - 스케치



- 메모리맵 주소 사용

```
// PB5 at port 13
```

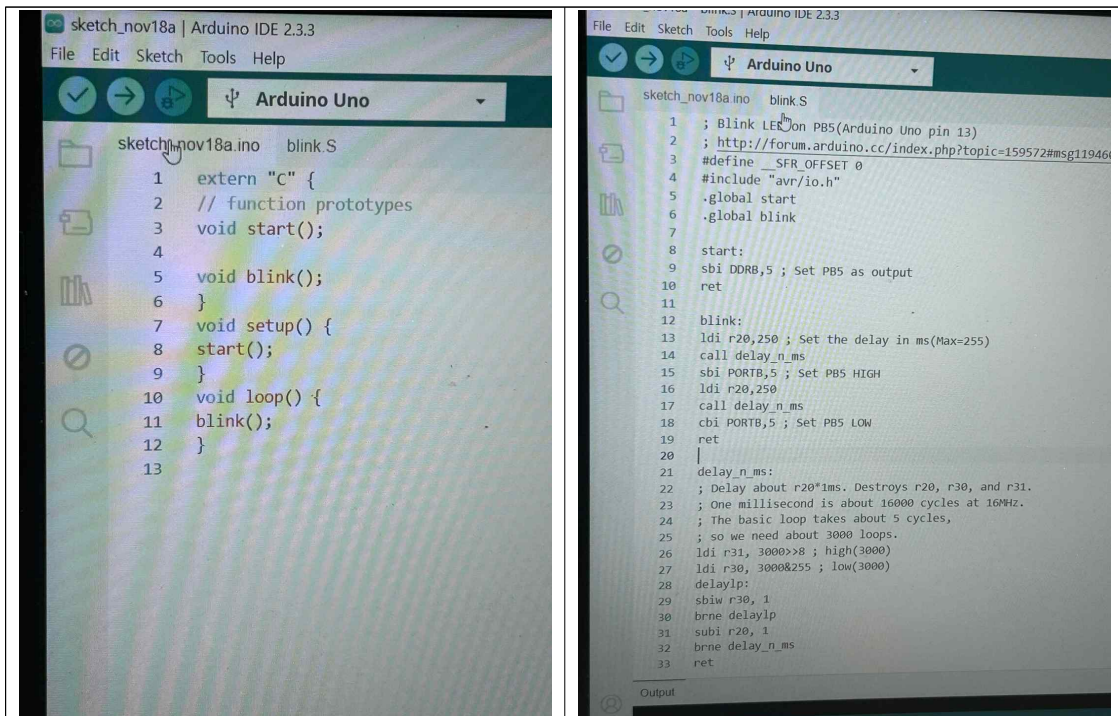
```
void setup() {  
  DDRB = 1<<5; // DDRB  
  
  void loop() {  
    PORTB |= 1<<5; // PORTB  
    delay(1000);  
    PORTB &= ~(1<<5); // PORTB  
    delay(1000);  
  }  
}
```

- AVR-GCC 인라인 어셈블리 사용

```
// PB5 at port 13
void setup() {
  asm("sbi 0x4, 5"); // DDRB = 0x20 + 4 = 0x24
}

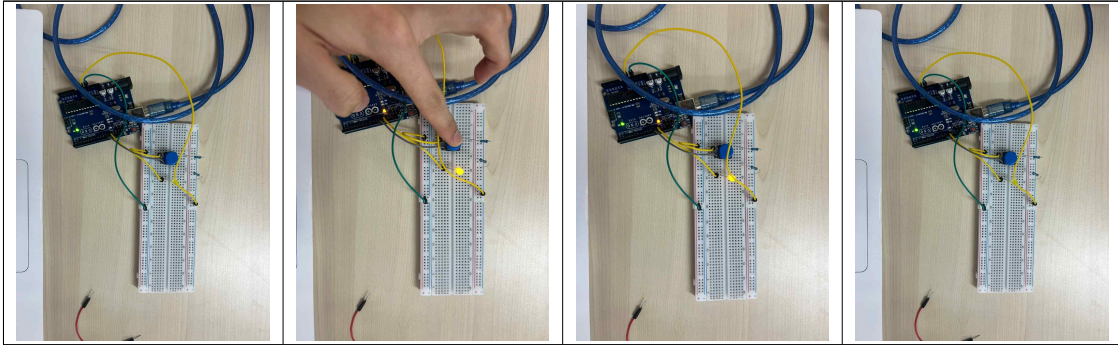
void loop() {
  asm("sbi 0x5, 5"); // PORTB
  delay(1000);
  asm("cbi 0x5, 5"); // PORTB
  delay(1000);
}
```

-어셈블리 소스파일 사용



▶ 10. 어셈블리 언어를 이용한 버튼으로 LED를 제어

※ 버튼을 누르면 불이 들어오고 버튼을 누르지 않아도 불이 남아 있다 시간이 지나면 불이 꺼지게 된다



메모리맵 주소 사용

AVR-GCC 인라인 어셈블러 사용

<pre>// PB4 at port 12, PB5 at port 13 void setup() { DDRB = 1<<5; DDRB &= ~(1<<4); } void loop() { int press = (PINB >> 4) & 0x1; if (press == 1) PORTB = 1<<5; else PORTB &= ~(1<<5); delay(100); }</pre>	<pre>// PB4 at port 12, PB5 at port 13 void setup() { asm("sbi 0x4, 5"); // DDRB, PB5 asm("cbi 0x4, 4"); // DDRB, PB4} void loop() { volatile int status; asm volatile ("ldi %0, 1 \n" //high "sbis %1, %2 \n" //skip next if pin high "clr %0 \n" //low : "=r" (status) : "I" (0x3), "I" (4) // 출력, PINB, PB4); if (status == 1) asm("sbi 0x5,5"); //PORTB else asm("cbi 0x5,5"); //PORTB delay(1000); }</pre>
---	---

▶ 0부터 인자까지 합산하는 LCD에 출력



```
sum.ino  sum.S
// sum.ino
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
extern "C" {
    int sum(int);
}
void setup() {
    // put your setup code here, to run once:
    lcd.init();
    lcd.backlight();
}
void loop() {
    // put your main code here, to run repeatedly:
    int v;
    v = sum(10);
    lcd.display();
    lcd.setCursor(0, 0);
    lcd.print("The summation is");
    lcd.setCursor(0, 1);
    lcd.print(v);
    delay(1000);
}
```

```
sum.ino  sum.S
sum.S
.text
.global sum
sum: // 단순 합산 반복. R24보다 크면 반복
ldi r18, 0
ldi r19, 0
loop:
add r18, r19
inc r19
cp r24, r19
brge loop
mov r24, r18
clr r25
ret
```

3. 결과

실습결과 검토 및 요점

(1) 실습 1번

정수형 변수를 선언하여 값을 할당 하고 pinMode 함수로 led핀을 전압출력모드로 설정하고 digitalWrite 함수로 led 핀에 HIGH전압을 출력 시킨다 delay 함수로 0.5초 동안 대기시켜서 0.5초 간격으로 불이 켜지고 꺼지는 것이 반복되는 것을 볼 수 있다.

(2) 실습 2번 7세그먼트

2차원 배열을 이용해 0~9까지의 숫자를 7세그먼트의 점 표현 제외 7개의 led를 이용하여 1~9까지 순차적으로 표현하였다. 캐소드와 애노드의 방식이 있어 그거에 맞는 방식으로 회로를 구상하고 코드를 만들어 나타내었다.

(3) 실습 3번

프로그래밍을 이용해 디지털 입력으로 led를 제어하는 실습이다.

if문과 digitalRead(key)로 버튼이 입력 되었는지 확인 할 수 있고 key1이 HIGH 이고 key2가 LOW인 경우엔 led1에만 불이 들어온다.

(4) 실습 5번 시리얼 통신

UNO와 USB포트를 이용해 컴퓨터와 아두이노를 연결하고 시리얼 함수를 이용하여, 아두이노 IDE에 있는 시리얼 모니터를 이용해 메시지를 전송하고 값을 확인할 수 있다. 또한 LED1과 LED2를 타이핑으로 ON OFF를 할 수 있다

(5) 실습 7번 가변저항을 이용한 LED 밝기 조절

가변저항의 범위는 0~1023이고 PWM범위는 0~255이다.

가변저항과 PWM의 범위를 맞춰 주어야 하기 때문에 4를 나누어 범위를 맞춰준다.

그러면 0~254까지의 값엔 불이 약함 켜지고 1023에 가까워질수록 불이 강하게 켜진다 또한 0인경우엔 불이 꺼지는 것을 알 수 있다.

(6) 실습 8번 PWM 펄스를 이용한 부저 연주

배열을 이용하여 각 배열에 도~시 까지의 음을 저장하고 그 음악의 계이름에 따라 tone 함수로 연주한다. 음의 지속시간과 음의 딜레이를 잘 조절하여 음의 길이, 침표등을 표현하여 단 순히 음만 맞추는 것이 아닌 노래를 만들었다.

(7) 실습 9번 어셈블리 언어를 이용한 LED제어

DDRB = 1<<5에서 B는 포트 번호, 5는 핀 번호이다. 코드에서 사용된 DDRB(0x24) 포트 B의 방향을 PORTB(0x25) 포트 B의 출력 값을 의미한다. PORTB |= 1<<5은 HIGH, PORTB &= ~(1<<5)은 LOW의 값을 나타내어 led의 불을 켜고 끄는데 이용한다.

(8) 실습 10번 어셈블리 언어를 이용한 LED제어

실습 9번에 스위치를 추가한 실습으로 포트B의 입력 값을 받고 조건문을 이용하여 스위치의 입력을 확인한다. 버튼을 누르면 불이 켜지고 손을 떼게되더라도 불은 그대로 켜진상태를 유지하다가 시간이 지나면 불이 꺼지는 현상이 일어난다.

(9) 실습 11번 어셈블리 언어를 이용하여 주어진 값까지 합산하여 LCD 출력

어셈블리어를 이용하여 0부터 10까지의 값을 더하여 55가 LCD에 출력되게 만들었다.

또한 LCD의 저항을 조절하여 출력된 문자를 선명하게 볼 수 있도록 만들었다.

실습 사진을 보면 The summation is 55 라는 값이 출력됨을 알 수 있다.

사진의 파란색 부분을 돌리면 글자의 선명도를 조절할 수 있다.

